FINAL CASE STUDY REPORT

FindIt

Campus Lost & Found Mobile Application



IT-WS07 · Mobile Application Technology

College of Information and Communications Technology (CICT)

Nueva Ecija University of Science and Technology (NEUST)

November 13, 2026

GROUP MEMBERS

John Carlo DG. Lemence · Chris Karl M. Lacanilao

Manny N. Agulto · Reyven B. Calara

Repository: github.com/jsoul-dev/find-it

APK: GitHub Release v1.0 · FindIt-v1.0.apk

Video presentation: Google Drive folder

1 Project Information

Title	**FindIt: Campus Lost & Found Mobile Application**
Course	IT-WS07 Mobile Application Technology · CICT, NEUST
Deliverables	GitHub Repository | Direct APK Download (GitHub Release v1.0) | Video Presentation (Drive)

MEMBER NAME	STUDENT ID	MEMBER NAME	STUDENT ID
John Carlo DG. Lemence	SUM2023-01395	Manny N. Agulto	SUM2023-01888
Chris Karl M. Lacanilao	SUM2023-01362	Reyven B. Calara	SUM2023-02252

2 Problem and Proposed Solution

THE CAMPUS PROBLEM

In academic environments such as the NEUST campus, lost and found belongings (student IDs, tumblers, umbrellas, calculators, lab equipment) are traditionally managed through word of mouth, physical bulletin boards, or unorganized social media posts. This informal approach has three shortcomings:

1 No geographic context
A post saying "found near the canteen" does not say which bench or area, so students search blindly.

2 Ambiguous, no visuals
Text alone (e.g., "blue umbrella") leads to several students claiming one item, or owners not recognizing their property.

3 Delayed resolution
Without structured status tracking, students cannot tell whether an item is still missing or already returned.

INTENDED USERS

- **Students and faculty:** everyday campus occupants who lose personal items or find unattended belongings.
- **Campus security personnel and department staff:** guard houses and student affairs offices that record turned-over items, attach visual evidence, and mark items "Claimed" upon return.

PROPOSED SOLUTION

FindIt is a focused cross-platform mobile app built with **Ionic**, **Angular Standalone**, and **Capacitor**. It provides a shared Lost & Found registry connected to a cloud-hosted REST API.

> **Instant reporting with geographic pinning.** Posts carry a description, contact info, and an exact GPS pin from Capacitor Geolocation and an interactive Leaflet map.

> **Visual verification and cloud image hosting.** Photos are taken with Capacitor Camera and uploaded to the **ImgBB** CDN; the returned HTTPS image URL is saved in the MockAPI record. The device also caches the photo in `localStorage` for offline and low-bandwidth viewing.

> **Shared registry.** Reports sync with an HTTPS REST API (**MockAPI.io**), so every device sees the same items.

> **Real-time search and filtering.** An `ion-searchbar` searches name, description, and location; Lost / Found / Claimed filters update instantly through Angular signals.

> **Tactile feedback.** Capacitor Haptics signals success, validation warnings, and network or hardware errors.

3 Application Features & User Flows

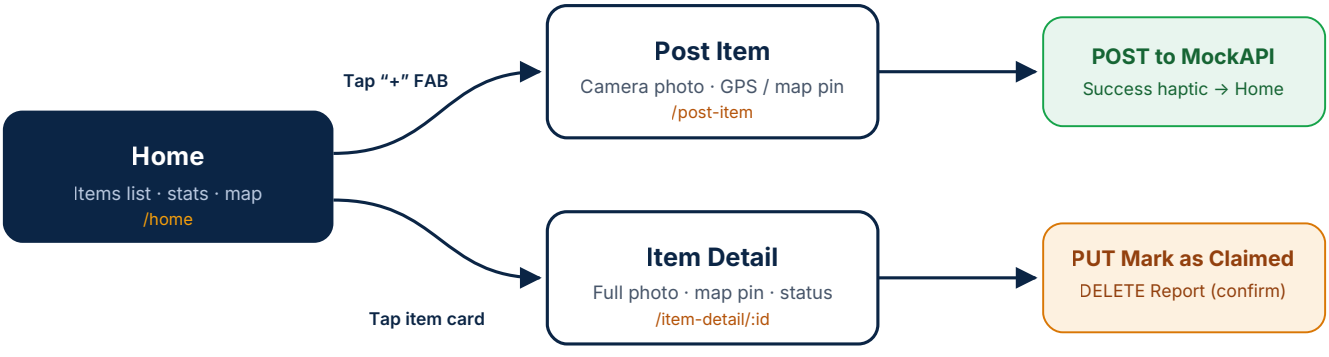


Figure 1. The app's three primary views and how users move between them.

Home / Items List `/home`

- **Statistics dashboard:** four cards (Total, Lost, Found, Claimed) driven by `computed()` signals; each card doubles as a filter toggle with keyboard navigation and high-contrast indicators.
- **Collapsible campus map:** OpenStreetMap view centered on NEUST Sumacab Campus (`15.44825, 120.93922`), toggled by "View Map / Hide Map".
- **Search and filter:** debounced `ion-searchbar` across names, descriptions, and contact/location notes, plus an `ion-segment` for All / Lost / Found / Claimed.
- **Live item cards:** reusable `ItemCardComponent` with photo thumbnail, name, description, contact, type badge (`LOST` `FOUND`) and status (Open or Claimed).
- **System states:** `ion-spinner` while loading, an empty-state message when no items match, and an error card with **Retry** if the network fails. A section divider shows a live count badge ("X items").
- **Navigation:** a floating action button routes to `/post-item`.

Post Item Page `/post-item`

- **Report form:** `[(ngModel)]` fields for report type (Lost or Found), item name, description, and contact information.
- **Photo capture:** "Attach Item Photo" opens Capacitor Camera (camera or photo library) with an immediate preview and a remove option.
- **"Locate Me":** Capacitor Geolocation reads the device GPS, recenters the Leaflet map, and drops a single `circleMarker`. **Manual fallback:** tap the map to drop or move the pin.
- **Client-side validation:** name, contact, and coordinates must be non-empty before the HTTP POST; failure shows a warning message and a haptic warning.

Item Detail Page `/item-detail/:id`

- **Full details:** large photo, name, description, contact, status badge, and a focused Leaflet map with a marker popup at the saved coordinates.
- **Lifecycle management (full REST CRUD):** for `Open` items, **Mark as Claimed** sends an HTTP `PUT`. Obsolete, duplicate, or test entries can be removed with **Delete Report** (toolbar trash icon or outline danger button), which sends an HTTP `DELETE` behind an `ion-alert` confirmation and haptics.

4 Technical Implementation

ARCHITECTURE OVERVIEW

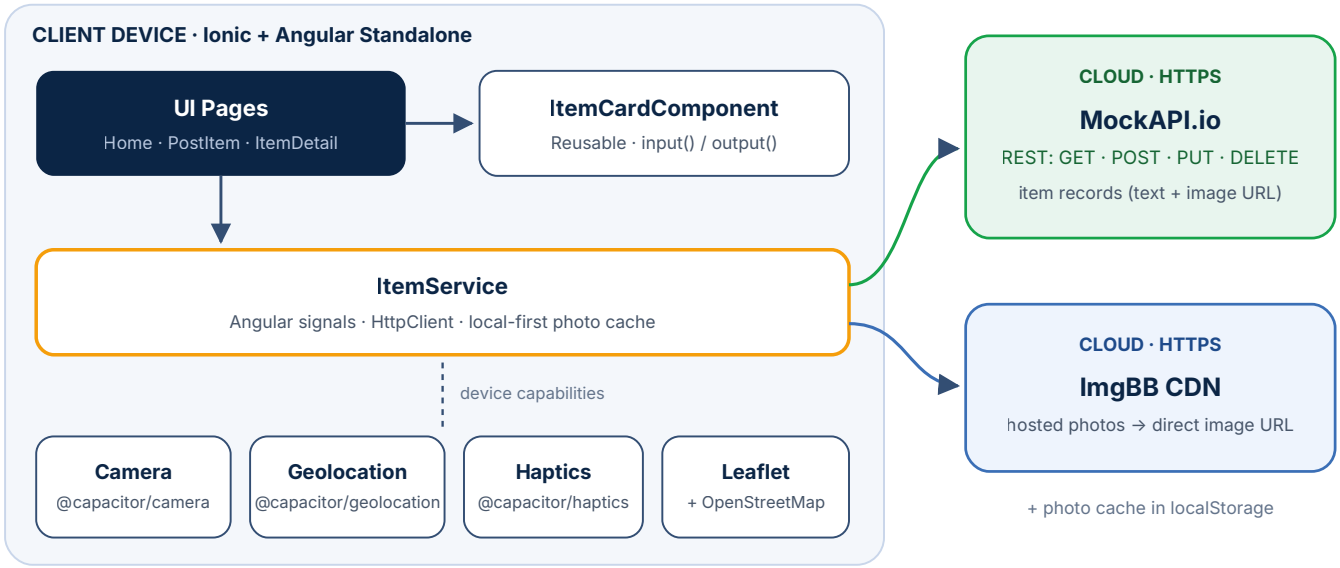


Figure 2. Architecture: client layers, Capacitor plugins, and the hybrid storage layer.

STACK AND ARCHITECTURAL ROLES

Ionic Framework (v9)	Mobile-first UI: `ion-header`, `ion-toolbar`, `ion-content`, `ion-card`, `ion-segment`, `ion-button`, `ion-fab`, `ion-badge`, `ion-img`
Angular (Standalone)	No `NgModule`; signal-based state (`signal`, `computed`); control flow `@for` / `@if`; `inject()` dependency injection
Capacitor (v8)	Three hardware plugins: `@capacitor/camera` (`Camera.getPhoto()`, PWA browser fallback), `@capacitor/geolocation` (`getCurrentPosition()`), `@capacitor/haptics` (`Haptics.notification()`)
Leaflet (v1.9)	Interactive map engine with OpenStreetMap tiles
MockAPI.io	Cloud-hosted HTTPS REST API giving real-time persistence across devices, with no local XAMPP/PHP server

CORE CODE SNIPPETS

1. Camera capture and cloud CDN upload (`ItemService`, `post-item.page.ts`)

```
uploadImage(dataUrl: string): Observable<string> {
  const cleanBase64 = dataUrl.replace(/^data:image\/\w+; base64,/, '');
  const formData = new FormData();
  formData.append('image', cleanBase64);

  return this.http
    .post<{ data: { url: string }; success: boolean }>(
      `${environment.imgbbUrl}?key=${environment.imgbbApiKey}`,
      formData,
    )
    .pipe(map((res) => res.data.url));
}
```

To avoid base64 payload overhead on MockAPI, photos are resized to 800 px width and uploaded to ImgBB, which returns a permanent HTTPS URL (e.g., `https://i.ibb.co/xyz/image.jpg`, about 35 bytes). Saving only that URL gives cross-device visuals at full resolution and bypasses MockAPI's 100 KB JSON payload ceiling.

2. Hybrid local-first photo persistence (`services/item.ts`)

```
saveLocalPhoto(id: string, dataUrl: string): void {
  try { localStorage.setItem(`findit_photo_${id}`, dataUrl); } catch {}
}

getPhoto(item: Item): string | undefined {
  try {
    const local = localStorage.getItem(`findit_photo_${item.id}`);
    if (local) return local;
  } catch {}
  return item.photo;
}
```

Full photos are cached locally by item ID while only compact payloads sync to the REST backend.

3. Geolocation and single-marker logic (`post-item.page.ts`)

```
async locateMe(): Promise<void> {
  try {
    const position = await Geolocation.getCurrentPosition();
    const lat = position.coords.latitude;
    const lng = position.coords.longitude;
    this.map.setView([lat, lng], 17);
    this.updateMarker(lat, lng);
    this.formError.set('');
  } catch {
    this.formError.set('Could not access GPS. Please check permissions or tap the map.');
    await Haptics.notification({ type: NotificationType.Error });
  }
}
```

One `circleMarker` is updated rather than duplicated; a denied permission or GPS failure shows a message and an error haptic.

4. `ItemService` with signals and HttpClient (`services/item.ts` , abridged)

```
@Injectable({ providedIn: 'root' })
export class ItemService {
  private http = inject(HttpClient);
  private url = environment.apiUrl;
  items = signal<Item[]>([]);
  loading = signal<boolean>(false);
  error = signal<string>('');

  fetchItems(): void {
    this.loading.set(true);
    this.error.set('');
    this.http.get<Item[]>(this.url).subscribe({
      next: (res) => { this.items.set([...res].reverse()); this.loading.set(false); },
      error: async () => { /* error message + error haptic */ },
    });
  }
  addItem(newItem: NewItem): Observable<Item> {
    return this.http.post<Item>(this.url, newItem);
  }
  markClaimed(item: Item): Observable<Item> {
    return this.http.put<Item>(`${this.url}/${item.id}`, { ...item, status: 'Claimed' });
  }
  deleteItem(id: string): Observable<Item> { /* DELETE, then update the items signal */ }
}
```

The service owns the shared state (`items` , `loading` , `error`) and exposes the REST operations used by the pages.

5 Testing and Evaluation

TEST TABLE EXECUTION RESULTS

#	FEATURE / TASK TESTED	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	**Open app, load items (GET)**	Items load from MockAPI with Lost/ Found badges and map renders	All seed items loaded immediately; map rendered at campus coordinates	Pass
2	**API unreachable (Airplane Mode)**	Error container with retry button appears; app does not crash	Error text displayed with a primary “Retry” button; error haptic fired	Pass
3	**Filter: Lost / Found / Claimed**	List dynamically updates to show only matching records	Segment switches instantly update the displayed cards with zero lag	Pass
4	**Post with empty name/ contact**	Form displays error message and triggers warning haptic	Error text displayed; phone vibrated with Warning haptic; no POST made	Pass
5	**Locate Me (permission granted)**	Map centers on user’s GPS; single circleMarker dropped	Map smoothly animated to device GPS location and pinned marker	Pass
6	**Locate Me (permission denied)**	Error message displayed and error haptic triggered	Prompt handled gracefully; error banner shown; map remains interactive	Pass
7	**Attach photo via Camera / Gallery**	Camera/Gallery prompt opens; captured image renders in preview	Photo snapped, thumbnail previewed; remove button functions correctly	Pass
8	**Submit valid item with photo (POST)**	Success haptic vibrates; item added to API with local photo cache	Success haptic fired; routed to Home; new item appears with photo thumbnail	Pass
9	**Open Item Detail page**	Details, attached photo, and focused Leaflet map marker display	Full photo, metadata, and map centered on exact coordinates loaded	Pass
10	**Mark as Claimed (PUT)**	Status badge updates to “Claimed” and Claim button disappears	Status updated on cloud API and UI badge changed to Claimed immediately	Pass
11	**Delete Report (DELETE)**	Confirmation modal opens; confirming purges item and navigates to Home	Alert presented; on confirm, item deleted from MockAPI, success toast shown, list refreshed	Pass

AUTOMATED UNIT AND END-TO-END TESTS

7 / 7

Vitest unit tests `npm run test:unit`

7/7 passing unit test suites validating `ItemService` (GET, POST, PUT, DELETE, signal state updates), `ItemDetailPage` , `PostItemPage` , `HomePage` , and `ItemCardComponent` .

4 / 4

Playwright E2E tests `npm run test:e2e`

Run in simulated mobile Chrome (**Pixel 7** viewport): (1) Home header and the four segment buttons; (2) Leaflet `#map` mounts; (3) `/post-item` form cards, photo button, GPS trigger, and submit button; (4) live keyword search filtering across cards.

LIMITATIONS AND USABILITY ISSUES

1. **External image hosting dependency and retention.** Photos live on the ImgBB CDN, with lightweight URLs stored on MockAPI. ImgBB retains uploads indefinitely on public endpoints and enforces a 32 MB per-file limit (FindIt adds a 10 MB client-side guard). Because this is an unauthenticated public prototype, users should photograph only ordinary property and avoid IDs, grade slips, passwords, or personal documents. If the image host is unreachable, FindIt falls back to `localStorage` caching so reports still go through.
2. **Duplicate entry policies.** *Submission concurrency:* accidental double-posts are blocked client-side by the `[disabled]="submitting()"` lock on the submit button. *Domain tolerance:* MockAPI has no unique constraints, and FindIt deliberately allows similar titles, since many students may lose or find identical items (black umbrellas, Casio calculators, PE uniforms) and strict deduplication would reject genuine reports.
3. **Unauthenticated public API.** MockAPI.io endpoints are publicly writable; any client with the URL can issue POST or PUT requests.
4. **Online map tiles.** OpenStreetMap tiles need internet access; offline, uncached regions show a blank grid.

PROPOSED IMPROVEMENTS (CO4)

1. Dedicated cloud storage
Upload original high-resolution photos to Firebase Storage or AWS S3 and save immutable image URLs to the database.

2. Authentication
Firebase Authentication / JWT with official school email (`@neust.edu.ph`) to prevent unauthorized posts.

3. Push notifications
Cloud Messaging alerts when an item matching a category or keyword is reported found near a user's department.

4. Offline map storage
Cache vector/raster tiles locally with SQLite or IndexedDB for full offline map browsing.

6 AI Usage Disclosure

In compliance with the assignment brief's guidelines on responsible AI use:

AI tools consulted	Claude Opus 4.6 (planning and architecture analysis); Gemini 3.8 Flash (implementation scaffolding)
Tasks assisted by AI	Formulating the 10-milestone project plan, configuring the Angular Standalone route structure, generating TypeScript scaffolding, and integrating Leaflet and Camera PWA elements.
Authored / tested by the group	Architecting the hybrid local-first photo storage strategy; configuring the MockAPI dashboard; testing Camera, GPS permissions, physical vibration, and map interactivity on real Android hardware; compiling the native Android project in Android Studio; and writing this report.
Security & privacy	No personal passwords, private tokens, or proprietary credentials were shared with any AI tool. The public MockAPI endpoint and ImgBB CDN upload are configured for academic showcase of mobile capabilities; users should not upload sensitive personal documents or private ID cards to the public prototype.

7 Installation Instructions (For Course Instructor)

1. **Download the APK:** get `FindIt-v1.0.apk` from GitHub Release v1.0.
2. **Allow sideloading:** on your Android device, enable **"Install unknown apps"** for your file manager or browser.
3. **Install:** tap `FindIt-v1.0.apk`. If Google Play Protect shows an *Unrecognized app* alert, tap **"More details"**, then **"Install anyway"**.
4. **Grant permissions:** tap **"Allow"** when prompted for Location and Camera access.
5. **Connectivity:** the app uses our live HTTPS MockAPI endpoint, so keep Wi-Fi or mobile data on while testing.

References and Project Links

PROJECT LINKS

- Source code: https://github.com/jsoul-dev/find-it
- Installable APK (v1.0): https://github.com/jsoul-dev/find-it/releases/download/v1.0/FindIt-v1.0.apk
- Video presentation: https://drive.google.com/drive/folders/191cjkX1alb3Y6yMHoddn8I2qd4HNR792

TECHNOLOGY REFERENCES

- Ionic Framework documentation: https://ionicframework.com/docs
- Angular documentation: https://angular.dev
- Capacitor documentation (Camera, Geolocation, Haptics plugins): https://capacitorjs.com/docs
- Leaflet API reference: https://leafletjs.com/reference.html
- OpenStreetMap tile usage and attribution: https://www.openstreetmap.org/copyright
- MockAPI.io: https://mockapi.io
- ImgBB API: https://api.imgbb.com

Course materials: ITWS07 Unit 1 (Foundation Review), Unit 2 (Device Geolocation and Interactive Map Integration), Mobile Sensors and Device Feedback, and the Final Case Study guide.